

4.3.1 MQTT介绍

4.3.2 MQTT原理

4.3.3 MQTT示例

■ MQTT协议

- MQTT (Message Queuing Telemetry Transport, 消息队列遥测传输协议), 是基于ISO标准框架的发布/订阅消息协议。
 - 基于TCP协议
 - 专为物联网设计



■ 协议架构

- 发布者(Publisher)、代理(Broker)、订阅者(Subscriber)
- 客户端 (Client) 、服务器 (Server)

客户端 (Client)

客户端是一个使用MQTT协议的程序或设备，它能够：

- 打开与服务器的网络连接；
- 发布其他客户端可能感兴趣的信息；
(作为发布者)
- 订阅它有兴趣接收的信息；(作为订阅者)
- 取消订阅以删除对信息的请求；
- 关闭与服务器的网络连接。

服务器 (Server)

服务器是在发布消息的客户端和进行订阅的客户端间充当中介的程序或设备，它能够：

- 接受来自客户端的网络连接；
- 接受客户端发布的消息；
- 处理来自客户端的订阅和取消订阅请求；
- 转发与客户端订阅匹配的消息；
- 从客户端关闭网络连接。

■ 工作流程

- 建立连接
- 发布订阅
 - 订阅者指定要订阅的Topic和QoS
 - 发布者将消息发送到Broker指定的Topic
 - Broker将消息发送给指定的订阅者（根据订阅关系）
- 消息确认
 - 根据QoS确认
- 关闭连接

■ 主题与主题过滤器

- MQTT中的应用消息通过主题名进行消息分类，主题名不需要创建，可以直接使用。主题过滤器是指含有通配符的主题，目的是让客户端同时订阅多个主题。在主题中，存在多个不同的通配符以供完成不同的功能：

主题级别分隔符 (/)

主题级别分隔符用于将主题名分隔为多个主题级别，如：

- sport/tennis/player1
- sport/tennis/player2

多级通配符 (#)

多级通配符可以涵盖任意数量的主题级别，为了确定匹配主题，多级通配符总为主题过滤器中的最后一个字符，并且它前面必须是一个主题级别分隔符，如sport/tennis/player1/#可含：

- sport/tennis/player1
- sport/tennis/player1/ranking
- sport/tennis/player1/score/wimbledon

单级通配符 (+)

单级通配符只匹配一个主题级别，可用在主题过滤器中的任何级别（包括第一级和最后一级）。在使用时，它必须占据过滤器的整个级别，且可以与多级通配符配合使用，如：

- +/tennis/#
- sport+/player1/ranking

特殊通配符 (\$)

以“\$”开始的每个主题都会被特殊对待，它们通常不会被包含在被订阅的内容中，服务器会防止客户端使用此类主题名称与其他客户端交换消息，这些主题被保留为MQTT代理服务器的内部特性，如：

- \$SYS/

- 服务质量 (Quality of Service, QoS)
 - QoS 0: 至多一次
 - QoS 1: 至少一次
 - QoS 2: 只有一次

■ 保活机制

- MQTT协议是承载于TCP协议之上的，而TCP协议以连接为导向，在连接双方之间，提供稳定、有序的字节流功能。但是，在部分情况下，TCP可能出现**半连接**问题。
- MQTT协议提供了**保活 (Keep Alive) 机制**，使客户端和MQTT服务器可以判定当前是否存在半连接问题，从而关闭对应连接。
- Keep Alive是CONNECT报文中的一个双字节整数，它是客户端完成传输一个MQTT报文到发送下一个报文之间允许经过的最大时间间隔。如果这个值不是零，并且在没有发送任何其他MQTT控制报文的情况下，客户端必须发送一个PINGREQ报文。否则经过这个时间段的**1.5倍**内没有收到任何报文，则它必须关闭与客户端的网络连接。



■ MQTT控制报文

- 通过交换一系列已定义的控制报文来运行MQTT协议
- 这些控制报文最多由三部分组成

MQTT数据包结构
固定头部 (Fixed header)
可变头部 (Variable header)
消息体 (Payload)

● 固定头

--Topic、 DUP flag、 QoS、 Retain、 Remaining Length

7	6	5	4	3	2	1	0
Topic				DUP flag	QoS		Retain
Remaining Length							

软件学院 · 工业互联网实验

- 固定头

--Topic

--DUP flag

--QoS

--Retain

--Remaining Length

7	6	5	4	3	2	1	0
Topic				DUP flag	QoS		Retain
Remaining Length							

4.3.2 MQTT原理

■ MQTT报文

- 固定头

- Topic

- DUP flag

- QoS

- Retain

- Remaining Length

数值	二进制表示	含义
0	00	至多一次，发完即丢弃
1	01	至少一次，需要确认回复
2	10	只有一次，需要确认回复
2	11	保留待用

76543210

TopicDUP flagQoSRetain

Remaining Length

- 固定头

--Topic

--DUP flag

--QoS

--Retain

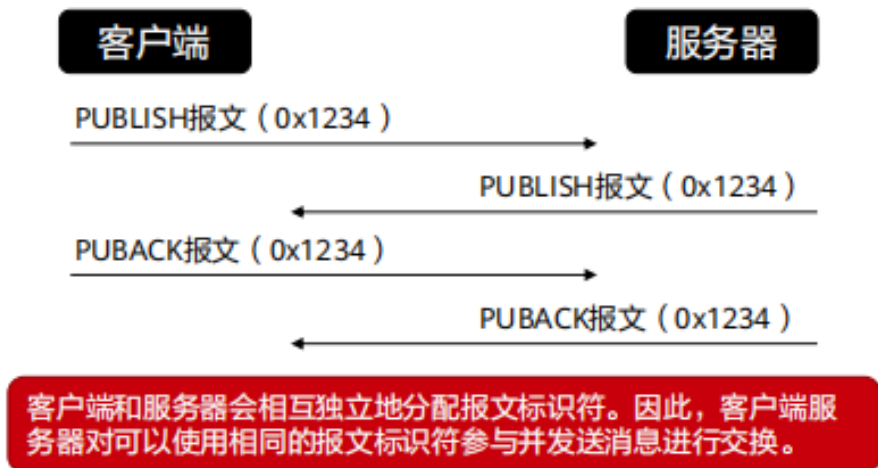
--Remaining Length

7	6	5	4	3	2	1	0
Topic				DUP flag	QoS		Retain
Remaining Length							

■ MQTT报文

● 可变头：报文标识符

- 一部分类型的报文包含可变报头的部分，它位于固定报头和数据载荷的中间位置。可变报头中包含的内容通常与报文类型密切相关，**报文标识符**是在可变报头中常见的部分。
- 报文标识符是一个长度为两字节的整型数值，它将用于客户端和服务端进行报文交互时标识对应的报文。



包含报文标识符的报文	说明
PUBLISH	包含 (当QoS>0时)
PUBACK	包含
PUBREC	包含
PUBREL	包含
PUBCOMP	包含
SUBSCRIBE	包含
SUBACK	包含
UNSUBSCRIBE	包含
UNSUBACK	包含

■ MQTT报文

● 可变头：属性

- MQTT协议可变报头的最后一组字段是属性，除了**PINGREQ**和**PINGRESP**报文外，其他类型的报文都可以包含此字段。同时，在CONNECT报文中，带有载荷的报文还会在遗嘱属性字段中包含一组可选的属性。
- 属性由定义其用法和数据类型的标识符组成，后面会跟随一个值。如果某一类型的控制报文与其包含的属性标识符不符，则表示该报文格式错误。

部分属性标识符及其用途

属性标识符		名称（用途）	类型	报文类型
DEC	HEX			
1	0x01	载荷格式指示符	1字节	PUBLISH
3	0x03	消息过期间隔	4字节	PUBLISH
8	0x08	响应主题	UTF-8编码字符串	PUBLISH
11	0x0B	订阅标识符	可变字节	PUBLISH, SUBSCRIBE
34	0x22	主题别名最大值	2字节	CONNECT, CONNACK
36	0x24	最大QoS	1字节	CONNACK
39	0x27	最大报文大小	4字节	CONNECT, CONNACK

■ MQTT报文

- 载荷

- 整体：对部分控制报文来说，载荷是最后一部分

- 例如：对PUBLISH来说，载荷就是前述的应用消息

本例介绍在Windows中使用python实现MQTT客户端功能的相关知识。

■ 环境配置

- ① 代理的安装：此处用EMQX Broker（开源MQTT消息服务器），安装过程有平台差异，参照<https://www.emqx.io/docs/zh/v4.3/getting-started/install.html>官网教程即可。
- ② 测试与调试：成功启动EMQ后，可通过浏览器访问[http://localhost:18083 admin/public](http://localhost:18083/admin/public)进入EMQ控制台，在工具 > Websocket模块进行客户端建立连接、订阅主题、发布消息、接收消息等测试和调试工作。
- ③ 配置客户端：在命令行中用 `pip install paho-mqtt` 安装MQTT客户端支持库paho-mqtt。

■ 创建步骤

创建MQTT客户端的一般步骤为：

- ① 创建一个客户端实例；
- ② 使用任一connect*()方法连接到MQTT代理；
- ③ 调用任一loop*()方法保持与MQTT代理通信；
- ④ 使用subscribe()方法订阅一个主题并接收消息；
- ⑤ 使用publish()方法向MQTT代理发布消息；
- ⑥ 使用disconnect()中断与MQTT代理的连接。

4.3.3 示例

■ 函数介绍

➤ 构造方法

```
Client(client_id="", clean_session=True, userdata=None, protocol=MQTTv311,  
        transport="tcp")
```

➤ connect*()

```
connect(host, port=1883, keepalive=60, bind_address="")//客户端阻塞连接MQTT代理
```

```
connect_async(host, port=1883, keepalive=60, bind_address="")
```

4.3.3 示例

■ 函数介绍

➤ loop*()

```
loop(timeout=1.0)//定期调用处理事件
```

```
loop_start()  
loop_stop(force=False)
```

```
loop_forever()
```

■ 函数介绍

➤ subscribe()

```
subscribe(topic, qos=0)//订阅一个或多个主题
```

该方法有三种不同的调用方式：

```
# 1. 参数为字符串和整数, 订阅topic1
subscribe("my/topic1", 2)
# 2. 参数为字符串和整数元组, 订阅topic2
subscribe(("my/topic2", 1))
# 3. 参数为字符串和整数元组的列表. 订阅topic1和topic2
# 单次调用多个主题, 比多次调用subscribe更有效
# 下面这行代码相当于先后执行1和2
subscribe([("my/topic1", 2), ("my/topic2", 1)])
```